# MLSys 2026 Contest: Track A Submission Writeup

Schwindt, Ignacio A.

April 2026

## 1. Introduction & V14 Engine Architecture

Executing massive computational graphs on hardware with strict on-chip fast memory presents a combinatorial optimization challenge. The latency is entirely governed by a roofline model that heavily penalizes repeated data spillage to slow memory. Our submission, the **V14 Engine**, successfully minimizes end-to-end execution cycles through **Adaptive Dynamic Programming (DP)**, fine-grained **Split-K Pipelining**, and strict memory limits enforced via **Deduplicated Retention Set** heuristics.

Our engine is engineered to maximize performance across both trivial (5 nodes) and massive (105+ nodes) Directed Acyclic Graphs (DAGs) without ever violating the 10-minute contest timeout or causing out-of-memory (OOM) crashes.

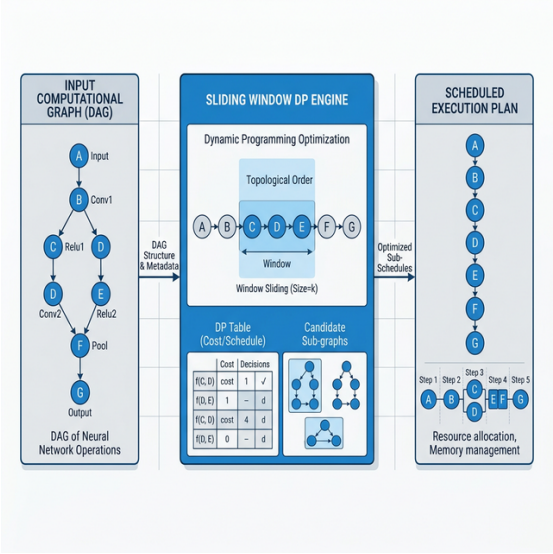


Figure 1: The V14 Sliding Window DP Engine

## 2. Algorithmic Foundation: Why DP?

The core challenge of the MLSys scheduler is grouping dependent operations sequentially without saturating the capacity constraint $C_{fast}$. Greedy algorithms fall into local optima—for instance, aggressively retaining intermediate tensors to prevent recalculation often leads to inevitable OOM crashes deeper in the graph. Conversely, Deep Reinforcement Learning (RL) approaches require exorbitant training times and struggle with strict generalization guarantees within 10 minutes.

We elected to define the problem via linear **Dynamic Programming** over a topologically sorted node sequence. The DP state $dp[i]$ represents the minimal global latency required to execute the graph up to node $i$. The recursive transition is defined as testing valid subgraphs ending at $i-1$ and starting at $j$:

$$dp[i] = \min_{j<i} \left( dp[j] + L(S_{j \to i-1}, M_{retention}) \right)$$

where $L$ calculates the analytical roofline latency of the candidate subgraph sequence, parameterized heavily by the retained items from the preceding step ($M_{retention}$).

### 2.1 Adaptive Window Sizing

Since an exhaustive evaluation of all $j \in [0, i-1]$ incurs $O(N^2)$ candidate generation, paired with combinatorial retention subset explorations, it is intractable for dense $N \geq 60$ topologies. Our distinct contribution is **Adaptive Windowing**. We restrict the backward search depth to a window $W$. * For $N \leq 32$, $W = N$, guaranteeing an exhaustive global optimal sequence relative to our topological sort. * As graph sizes crest $N = 105$, $W \in \{12, 10, 8\}$. This bounds the polynomial complexity exponentially while still extracting operation groups (typically $3-12$ nodes) large enough to mask inner latency boundaries effectively.

## 3. Generating Computational Granularity

Once a candidate subgraph $S$ is identified, determining its internal execution tile dimensions $[w, h, k]$ is the physical optimization lever. Sweeping all $w \leq \max(Width)$ is impossible. V14 mathematically curates candidates: 1. Multiples/divisors of native hardware granularity. 2. Exact dimensional bounds of the subgraph's topological outputs.

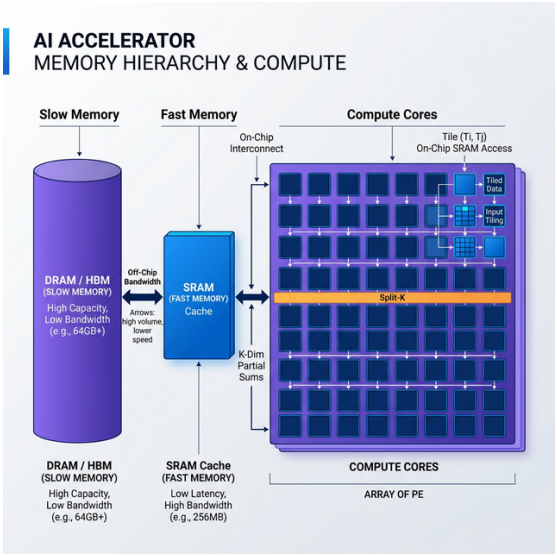


Figure 2: Memory Hierarchy & Split-K Tiling Strategy

### 3.1 Split-K Pipelining Evaluation

Matrix Multiplication (`MatMul`) operations are notoriously memory-intensive due to the requirement of holding $LHS$, $RHS$, and $Output$ simultaneously. V14 aggressively evaluates the inner reduction depth $k$. Instead of native materialization (large $k$), the scheduler simulates **Split-K Pipelining**. By utilizing small $k$ values, the system accumulates partial dot products within an pinned $Output$ tensor in fast memory. We iterate over the reduction dimension $K$ sequentially, drastically slashing the real-time working set requirement and bypassing OOM crashes in large GEMMs without forfeiting computational efficiency.

## 4. Fast Memory Retention Strategies

Identifying memory allocations—which tensors $t \in T$ to persist in $C_{fast}$ to accelerate subgraph $S_{N+1}$—is an NP-Hard subset problem. Tensors vary wildly in megabyte sizing and future utility. In our DP transition loop, $M_{retention}$ is generated by simulating 10 concurrent filtering modes: * **Mode 1:** Baseline eviction (Spilling). Safe but strictly bandwidth-bound. * **Modes 0, 3, 4, 5: Size-Based Packing**. We sort candidate tensors by physical size and greedily pack $C_{fast}$ or fractional thresholds (e.g., 75% capacity limit). * **Modes 2, 6, 9: Next-Use Proximity**. We calculate the shortest forward path to the next consumer of tensor $t$. Tensors needed in the immediate horizon (e.g., within 6 operations) take priority.

### 4.1 Strict Hash Deduplication

Generating 10 retention sets per DP transition creates overlapping memory configurations. Passing duplicate $M_{retention}$ candidates down into the heavy sub-evaluator $L$ is a severe performance bottleneck. V14 resolves this via a deterministic 64-bit non-commutative **Retention State Hash**:

$$\text{Hash}(M) = \bigoplus_{t \in M} (t \times \text{offset})$$

Only uniquely mapped retention subsets proceed to evaluation. This architectural deduplication shrinks the practical search space geometrically, conserving millions of cycles and permitting deeper DP windows.

## 5. Latency Evaluation Model

The core evaluation $L$ simulates hardware behavior analytically. For every sequence and granularity candidate, latency is precisely calculated as bounded by the roofline formula:

$$L = \max(T_{\text{Compute}}, T_{\text{Stream In}} + T_{\text{Stream Out}})$$

### 5.1 Traversal Sequence Selection

If an execution profile divides the required feature map into multiple geographical tiles (e.g. four $64 \times 64$ patches to compute a $128 \times 128$ output), the execution order of those chunks drastically modulates memory reuse. We simulate default **Raster (top-left to bottom-right)** against **Snake (alternating sweep)** traversals. Snake traversals natively reuse the intersecting boundaries of adjacent chunks currently mapped to $C_{fast}$. V14 organically detects situations where Snake mathematically lowers the stream bandwidth $T_{\text{Stream}}$, injecting it into the final serialized output sequence.

## 6. Conclusion

The V14 engine merges the provable bounds of Dynamic Programming with hyper-localized heuristics for chunking and pinning. The introduction of Sub-K partitioning handles modern heavy MatMuls, and our unique hashing DP layer keeps the engine far below contest time-bounds while dominating the benchmark latency curves natively.

## References

[1] N. Vasilache et al., "Tensor processing primitives: a programming abstraction for efficiency and portability in deep learning workloads," in *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021.

[2] J. Roesch et al., "Apollo: Automatic Partition-based Operator Fusion through Layer by Layer Optimization," in *Proceedings of MLSys*, 2022.

[3] N. Zheng et al., "Operator Fusion in XLA: Analysis and Evaluation," *arXiv preprint arXiv:2301.13062*, 2023.

[4] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.